# 1. The Shell: Prompts, Commands, Arguments, Exit Codes

## 01. The Shell: Prompts, Commands, Arguments, Exit Codes

**Phase:** Phase 1 — Shell and filesystem
**Estimated time:** 1 day
**Slug:** `01-shell-basics`

### Learning objectives

- Run commands with arguments and flags
- Read exit codes (`$?`) and why they matter
- Use `--help` / man pages productively
- Distinguish absolute vs relative paths

### Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

### Why this matters

Every task on a Linux server starts at a prompt. Understanding how the shell splits a line into a command and its arguments, and how a program reports success or failure through an exit code, is the foundation for all scripting, CI pipelines and debugging. A misunderstood quote or ignored exit status is behind many production incidents.

### Core concepts

The prompt, for example `student@box:~/work$`, shows user, host, current directory (`~` means home) and a final character: `$` for a normal user, `#` for root. The shell reads a line, splits it into *words* on unquoted whitespace, expands things like `$HOME`, `*.txt` and `$(date)`, then runs the first word as the command and the others as arguments.

```
ls  -l  "my file.txt"  /tmp
|   |        |           |
|   |        |           +-- argument 3
|   |        +-- argument 2 (quotes keep the space)
|   +-- option (argument 1 by convention)
+-- command (argv[0])
```

Commands come in kinds: builtins (`cd`, `echo`), external programs (`/usr/bin/ls`), aliases and functions. `type NAME` tells you which.

When a command ends it returns an *exit code* from 0 to 255. By convention 0 means success and anything else means failure. The code of the last command is in `$?`.

| Code | Usual meaning |
|---|---|
| 0 | Success |
| 1 | General error |
| 2 | Misuse of the command (bad option) |
| 126 | Found but not executable |
| 127 | Command not found |
| 128+N | Killed by signal N (130 = Ctrl-C, 137 = KILL) |

Quoting rules: single quotes keep text literal; double quotes allow `$VAR` and `$(cmd)` expansion but prevent word splitting; a backslash escapes one character.

Chaining: `a && b` runs b only if a succeeded; `a || b` runs b only if a failed; `a ; b` runs both regardless.

## Worked examples

Exit codes in action:

```
true; echo $?
# sample output: 0
false; echo $?
# sample output: 1
ls /nonexistent; echo $?
# sample output: ls: cannot access '/nonexistent': No such file or directory
# sample output: 2
nosuchcmd; echo $?
# sample output: bash: nosuchcmd: command not found
# sample output: 127
```

`true` and `false` are tiny programs that only return 0 and 1. GNU `ls` returns 2 for a serious trouble such as a missing argument path.

Arguments and quoting:

```
printf '[%s]\n' my file.txt
# sample output: [my]
# sample output: [file.txt]
printf '[%s]\n' "my file.txt"
# sample output: [my file.txt]
```

Without quotes the shell hands two arguments to printf; with quotes, one.

Conditional chaining:

```
mkdir demo && cd demo && echo "inside $(pwd)"
# sample output: inside /home/student/demo
grep -q root /etc/passwd && echo found || echo missing
# sample output: found
```

Identify a command:

```
type cd ls
# sample output: cd is a shell builtin
# sample output: ls is /usr/bin/ls
```

## Mini-lab

1. Run `echo $?` right after opening a shell; note it is 0.
2. Run `false`, then `echo $?`. Then run `echo $?` again and explain why it is now 0 (the previous command was echo).
3. Run `ls /does/not/exist; echo "code=$?"`.
4. Run `foo; echo "code=$?"` and confirm 127.
5. Create a script: `printf '#!/bin/bash\nexit 3\n' > t.sh`, run `./t.sh` (fails with 126 because it is not executable), then `chmod +x t.sh; ./t.sh; echo $?` prints 3.
6. Test quoting: `touch "a b.txt"; ls a b.txt` fails, `ls "a b.txt"` works.
7. Try `cd /tmp && pwd`, then `cd /nope && pwd` and observe that pwd does not run.
8. Use `type -a echo` to see both builtin and external versions.

## Decision guide / failure modes

Use `&&` when the second step depends on the first, `||` for fallbacks, and `;` only when steps are independent. Use single quotes for literal text (regexes, awk programs) and double quotes when you need variables. Use `$?` immediately; any other command overwrites it, so save it with `rc=$?` if needed later.

Common failures:

- Forgetting quotes around variables: `rm $file` with spaces or globs misbehaves; write `rm -- "$file"`.
- Assuming non-zero means a bug. `grep` returns 1 for “no match”, which is not an error.
- Pipelines hide failures: the status of `a | b` is b’s unless `pipefail` is set (module 7).
- Reading exit codes above 125 as normal; 127 and 126 are shell-level problems, 128+N means a signal.
- Using `;` where `&&` was meant, so a failed `cd` is followed by a destructive command in the wrong directory.

## Common mistakes

- Ignoring exit codes and assuming success
- Confusing flags that need spaces vs combined short flags
- Not reading man page SYNOPSIS

## Exercises

Work in `exercises/01-shell-basics/`. Read that folder’s `README.md` fully before coding.

```
cd exercises/01-shell-basics
chmod +x test.sh exercise.sh
./test.sh
```

**What you will build**

Implement two small functions. success_code() returns the exit code that conventionally means a command succeeded. failure_code() returns a typical generic failure exit code. Both return integers, not strings. success_code must be 0. failure_code must be a non-zero integer in the range 1..255; use 1, the general error code used by shell builtins and most tools. No arguments and no side effects are needed. The exercise checks that you know the convention behind “0 is success, non-zero is failure” that the shell relies on for && and ||.

**Study loop**

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

**Do not open `solutions/01-shell-basics/` until you have an honest failing attempt.**

## Self-check

Try to answer without scrolling up:

1. What does exit code 0 mean?
2. Absolute vs relative path?
3. How do you open the manual for `cp`?

## Further reading

- https://man7.org/linux/man-pages/man1/bash.1.html